

Setor de Educação Profissional e Tecnológica

Tecnologia em Análise e Desenvolvimento de Sistemas

DS141 - Estruturas de Dados II

03 - Recursão

Prof. Alex Kutzke

Maio de 2021

Roteiro

Definição

Propriedades

Exemplos

Recursão e TADs

Recursão

- Fundamental na matemática e na computação;
- Definição: “Algo definido em **termos de si próprio**”;
 - Matemática: Uma **função recursiva** é aquela que é definida em termos de si própria;
 - Computação: Um **programa recursivo** é aquele que “chama” (executa) a si mesmo.
- **Algoritmo recursivo**: aquele que resolve um problema a partir da solução de uma ou mais instâncias *menores* do mesmo problema (SEGEWICK; WAYNE, 2014);
 - Em C, utilizamos funções recursivas para implementar programas recursivos.

Relação de Recorrência

- Relações de recorrência são funções definidas recursivamente (matemática);
- Uma relação de recorrência é definida por duas partes:
 - conjunto de valores iniciais, ou bases; e
 - valores determinados em termos da própria função para valores **menores** de “entrada”.
- Talvez a mais conhecida seja a função *fatorial*:

$$N! = \begin{cases} 1, & \text{se } N = 0, \\ N * (N - 1)!, & \text{se } N \geq 1. \end{cases}$$

Implementação Recursiva Fatorial

```
1  int fat(int n)
2  {
3      if(n == 1) return(1);
4      return(n * fat(n-1));
5  }
```

- É sempre possível transformar um algoritmo recursivo em um não-recursivo:

```
1  for(t=1,i=1;i<=n;i++) t*=i;
```

- O inverso também é verdadeiro: é possível transformar qualquer algoritmo não-recursivo que envolva laços em algoritmos recursivos.

Análise Algoritmo Fatorial

- O algoritmo de fatorial apresentado ilustra as principais propriedades da recursão:
 - Possui uma condição de parada (término);
 - Executa ele mesmo para valores cada vez menores.
- Indução matemática pode ser utilizada para provar que o programa funciona como esperado:
 - Computa $0!$ (base da indução);
 - Sobre a suposição de que computa $k!$, para $k < N$ (hipótese indutiva), temos que o algoritmo computa $N!$.
- O mesmo raciocínio pode ser utilizado para resolver problemas mais complexos.

Condições

Em resumo, uma função recursiva precisa:

- Resolver pelo menos um caso base;
- Cada chamada recursiva deve envolver valores menores de argumentos.

Esses itens permitem acreditar que uma prova por indução é possível. Além de serem ótimos guias de implementação para algoritmos recursivos.

Vantagens e desvantagens

- Vantagens:
 - Deixa o código mais simples e mais compacto;
 - Nem sempre significa que é mais fácil de entender, mas sim que segue a definição matemática.
 - Fácil implementação (seguindo a recorrência matemática).
- Desvantagens:
 - Podem se tornar impraticáveis em termos de custo computacional;
 - É necessário fugir de implementações ruins.
 - Seu desempenho depende da implementação de pilhas do sistema operacional e da *profundidade da recursão*;
 - Em geral utilizam mais memória e são de difícil depuração.

Qual o problema dessa função?

```
1  int puzzle(int n)
2  {
3      if(n == 1) return(1);
4      if(n % 2 == 0)
5          return(puzzle(n/2));
6      else return(puzzle(3*n+1));
7  }
```

Máximo Divisor Comum

- Algoritmo de euclides:
 - Baseia-se na seguinte observação:
O máximo divisor comum de dois números inteiros x e y , com $x > y$, é o mesmo que o máximo divisor comum de y e $x \% y$.

Máximo Divisor Comum

- Algoritmo de euclides:
 - Baseia-se na seguinte observação:

O máximo divisor comum de dois números inteiros x e y , com $x > y$, é o mesmo que o máximo divisor comum de y e $x \% y$.

Um número t divide ambos x e y se, e somente se, t divide ambos y e $x \% y$, pois x é igual a $x \% y$ mais um múltiplo de y .

Algoritmo de Euclides Recursivo

```
1  int mdc(int m, int n)
2  {
3      if(n == 0) return(m);
4      return(mdc(n, m % n));
5  }
```

Avaliação de Expressões Pré-fixas

Como seria um algoritmo recursivo para expressões do tipo:

$* + 7 * * 4 6 + 8 9 5 ?$

Avaliador Pré-fixo

```
1 char a[100]; int i;
2 int eval(){
3     int x = 0;
4     while(a[i] == ' ') i++;
5     if(a[i] == '+') { i++;
6         return(eval() + eval()); }
7     if(a[i] == '*') { i++;
8         return(eval() * eval()); }
9     while((a[i] >= '0') && (a[i] <= '9'))
10         x = 10*x + (a[i++] - '0');
11     return(x);
12 }
```

Profundidade da Recursão

- A chamada de uma função carrega consigo um custo computacional “escondido”:
 - Pilha de chamadas de função.
- A **profundidade** de uma recursão é a quantidade de chamadas recursivas que a função realiza.

Algoritmo Fibonacci Recursivo

```
1  int fib(int n)
2  {
3      if(n == 0) return(0);
4      if(n == 1) return(1);
5      return(fib(n-1) + fib(n-2));
6  }
```

Quantas chamandas recursivas esse algoritmo faz quando $n = 25$?

O que se pode dizer sobre a complexidade desse algoritmo?

Recursão e TADs

- TADs baseadas em nós e ponteiros possuem, em geral, um caráter recursivo;
- Como contar o número de elementos de uma lista utilizando uma função recursiva?
- Como encontrar o maior elemento da lista?

O que esse algoritmo faz?

```
1  int aaaa(int a[],int l, int r){
2      int u,v;
3      if(l == r) return(a[l]);
4      int m = (l+r)/2;
5      u = aaaa(a,l,m);
6      v = aaaa(a,m+1,r);
7      if(u > v) return(u); else return(v);
8  }
9  ...
10 aaaa(v,0,tam_v);
```

O que esse algoritmo faz?

```
1  int aaaa(int a[],int l, int r){
2      int u,v;
3      if(l == r) return(a[l]);
4      int m = (l+r)/2;
5      u = aaaa(a,l,m);
6      v = aaaa(a,m+1,r);
7      if(u > v) return(u); else return(v);
8  }
9  ...
10 aaaa(v,0,tam_v);
```

- Encontrar o máximo do vetor;
- Abordagem dividir para conquistar.

Uma função recursiva que divide um problema de tamanho N em duas partes independentes (não vazias) que são resolvidas recursivamente, faz menos de N chamadas recursivas.

Programação Dinâmica

- Técnica de programação;
- Armazenar valores já calculados para chamadas recursivas;

```
1  int knownF[100];
2  int fib(int n)
3  {
4      int t;
5
6      if(knownF[n] != 0) return knownF[n];
7      if(n == 0) return(0);
8      if(n == 1) return(1);
9      t = fib(n-1) + fib(n-2);
10     return knownF[n] = t;
11 }
```

- Quantas chamadas recursivas esse algoritmo faz?

Referências utilizadas na apresentação I

 SEDGEWICK, R.; WAYNE, K. *Algorithms: Part I*. [S.l.]: Addison-Wesley Professional, 2014.